

Review toàn hệ thống VideoLevel (ZK-Stego)

Một lỗi media native, kênh CAVLC giao thức v3, proof camera ràng buộc video

Tài liệu kỹ thuật tiếng Việt

5 tháng 10 năm 2026

Cập nhật 05/10/2026. Bản review này được viết lại theo mã nguồn hiện tại trên nhánh `main` (kể cả các thay đổi chưa commit trong working tree). So với bản 03/10/2026, hai thay đổi lớn ngày 04/10/2026 được phản ánh: (1) *giao thức kênh v3*: khung chỉ còn `[0x03][độ dài 2 byte][payload]`, không còn MAC, nên payload trích ra chỉ là ứng viên cho tới khi proof Groth16 được kiểm; (2) *proof camera*: mạch `circuits/camera_video.circom` chứng minh một camera trong sổ đăng ký Merkle Poseidon xác nhận đúng video (hash toàn file trừ các bit carrier) và message; bên kiểm chứng chỉ cần gốc sổ đăng ký, verification key và khóa giấu tin, không cần secret của camera. Benchmark đã được chạy lại ngày 04/10/2026 với v3 + proof camera; bộ kiểm thử qua 142/142 trên 12 phase. Các phát hiện “proof không gắn với video”, “payload phát lại sang video khác vẫn được chấp nhận”, “verifier phải biết secret” và “không kiểm chứng công khai được” của bản trước đã được xử lý (mục 12). Nguồn đối chiếu: `doc/he_thong_hoat_dong.md` (mục 1, 8–12, 16, 17), `README.md`, `benchmark/NEW_BENCHMARKS.md`, `future_plan.md` và mã nguồn.

Mục lục

1	Phạm vi và cách đọc bản review	3
2	Hệ thống làm gì	3
3	Kiến trúc: một lỗi media và các thành phần quanh nó	3
4	Lớp zero-knowledge: proof camera ràng buộc video	4
4.1	Hai loại khóa và sổ đăng ký camera	4
4.2	Hash video và binding	5
4.3	Mạch <code>camera_video.circom</code>	5
4.4	Payload và nén proof	6
4.5	Xác minh fail-closed	7
4.6	Trusted setup	7
4.7	Proof chứng minh gì và không chứng minh gì	7
5	Kênh native CAVLC, giao thức v3	8
5.1	Ứng viên và lý do chọn đầu trailing-one	8
5.2	Tách khóa con bằng HKDF	8
5.3	Lịch, khung và whitening	9
5.4	Một giao thức theo đoạn và trích xuất mù	10
6	Native CLI và các sửa lỗi hardening	10
6.1	Lệnh và ranh giới tiến trình	10
6.2	Bộ đọc Annex-B từ stdin	10
6.3	Giới hạn tài nguyên và đường dẫn	11
7	Dịch vụ HTTP/WebSocket	11
7.1	Xác thực trước khi đọc body	11
7.2	Job verify với proof camera bắt buộc	12

7.3	Các bảo vệ khác	13
8	Bộ lập lịch realtime	13
9	Demo và kiểm thử	13
9.1	Một demo duy nhất	13
9.2	Bộ kiểm thử	14
10	Kết quả benchmark đã ghi (04/10/2026)	14
11	Điểm mạnh của hệ thống hiện tại	15
12	Các vấn đề đã xác minh và sửa trong các đợt review 02–04/10/2026	15
13	Contract giữa các bước: input, output và lỗi	17
14	Giới hạn còn lại	17
15	Kết luận	17

1 Phạm vi và cách đọc bản review

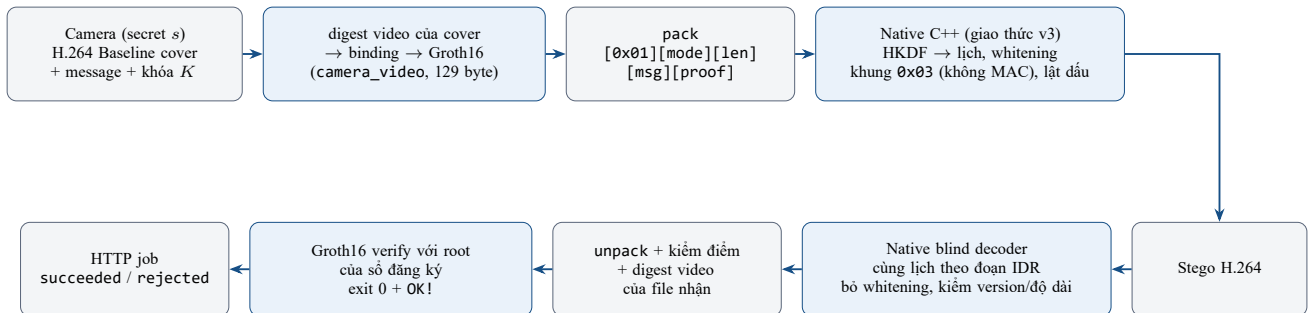
Đây là tài liệu review trạng thái hiện tại của hệ thống VideoLevel, không phải hướng dẫn chạy demo (hướng dẫn nằm ở README.md và demo/README.md). Trọng tâm là các thành phần runtime đang tồn tại: mạch camera_video, sổ đăng ký camera và hash ràng buộc video; cầu nối Groth16; lõi native C++ đọc/ghi H.264 CAVLC; kênh giấu tin giao thức v3; CLI native; dịch vụ HTTP/WebSocket; bộ lập lịch realtime; demo và bộ kiểm thử.

Các code listing được lấy trực tiếp từ source tree tại thời điểm biên soạn, kèm số dòng gốc để đối chiếu. Mỗi listing được đọc như review code: xác định precondition, theo dữ liệu từng bước, rồi kiểm tra output có đúng hợp đồng ở ranh giới kế tiếp hay không. Thư viện ngoài, mã sinh tự động, test và bảng VLC không được chép lại.

Mọi con số trong tài liệu lấy từ tài liệu và kết quả đã lưu trong repository (README, doc/he_thong_hoat_dong.md, benchmark/NEW_BENCHMARKS.md, benchmark/results/e2e_new.json, future_plan.md). Bản review này không chạy lại benchmark; số liệu benchmark là lần chạy đã ghi ngày 04/10/2026 (mục 10), và các số đo cũ hơn được ghi rõ là số liệu lịch sử.

2 Hệ thống làm gì

VideoLevel giấu một payload gồm message và proof Groth16 trong video H.264 Baseline bằng cách đặt dấu của một số hệ số trailing-one CAVLC trong các frame IDR, sau đó trích xuất mù bằng khóa giấu tin và bắt buộc xác minh proof. Proof chứng minh: *một camera có khóa nằm trong sổ đăng ký tin cậy (gốc Merkle công khai) đã xác nhận đúng video này và message này*. Bên kiểm chứng không cần secret của camera và không biết camera nào trong sổ đăng ký đã ký.



Hình 1: Luồng đầu–cuối hiện tại. Một lõi media native; Python điều phối proof, đóng gói, dịch vụ và demo.

Ở mức khái niệm, với cover V , message m , secret camera s , khóa giấu tin K và giới hạn $c = \max_bits_per_IDR$:

$$\begin{aligned}
 d &= \text{digest}_{K,c}(V), & b &= \text{SHA256}(\text{"proof-binding/v1"} \parallel \text{mode} \parallel d \parallel \text{SHA256}(m)), \\
 \pi &= \text{Prove}(s, \text{đường Merkle}; \text{root}, b_{hi}, b_{lo}), & B &= [0x01 \parallel \text{mode} \parallel |m| \parallel m \parallel \text{ser}(\pi)], \\
 (k_s, k_w) &= \text{HKDF}(K), & F &= [0x03 \parallel |B| \parallel B], & V_{stego} &= \text{embed}_{H.264}(F \oplus \text{KS}(k_w), \text{schedule}(k_s, c)), \\
 V_{stego}, K &\longrightarrow F \longrightarrow (\text{mode}, m, \pi), & d' &= \text{digest}_{K,c}(V_{stego}) = d, & & \text{Verify}(vk, \pi, (\text{root}, b_{hi}, b_{lo})).
 \end{aligned}$$

Native channel không tạo proof: caller tạo payload trước và C++ coi nó là dữ liệu opaque. Khóa không được nhúng trong H.264; bên nhận phải có cùng khóa giấu tin K 32 byte và cùng $\max_bits_per_IDR$. Secret camera s không bao giờ rời camera.

3 Kiến trúc: một lõi media và các thành phần quanh nó

Ba lớp có trách nhiệm tách biệt: *media* (bitstream sau khi nhúng vẫn hợp lệ, độ dài mã không đổi), *kênh giấu tin* (chỉ người có khóa tìm được vị trí và đọc được khung; từ v3 kênh không có MAC nên không tự phát hiện bit bị sửa) và *proof* (camera trong sổ đăng ký xác nhận đúng video và message; đây là lớp duy nhất xác thực payload).

Module	Trách nhiệm	Output cho downstream
native/include/zkstego/ cavlc_stream.hpp, native/ src/cavlc_stream.cpp	Annex-B, SPS/PPS, slice, macroblock, CAVLC; ứng viên dấu trailing-one; HKDF, lịch HMAC, khung v3, whitening; encoder/decoder theo đoạn IDR; hash video.	Stego Annex-B, payload ứng viên (chưa xác thực) hoặc digest video.

Module	Trách nhiệm	Output cho downstream
native/tools/blind_bits.cpp, native/tools/cli_io.hpp	CLI zkstego_blind_bits: nhúng/trích, video-digest; khóa qua stdin, file hoặc stdin/stdout, đường dẫn Unicode/dài trên Windows.	Exit code (2 = mọi lỗi), video, payload hex hoặc digest.
native/tools/inspect.cpp	Công cụ soi duy nhất zkstego_inspect: JSON, --summary, --macroblock, --segments.	Trace NAL/slice/ứng viên, lịch theo đoạn.
native/tests/cavlc_stream_tests.cpp	CTest native.	Bảng chứng traversal/rewrite.
circuits/camera_video.circom	$pk = \text{Poseidon}(s)$, đường Merkle 16 tầng tới root, hai nửa binding; 8 737 constraint.	Constraint system, 3 public input.
src/camera_registry.py, src/poseidon.py	Cây Merkle Poseidon độ sâu 16 (tối đa 65 536 camera), Poseidon với hằng số circomlib.	Root tin cậy, đường Merkle cho prover.
src/video_binding.py	Bản tham chiếu Python của hash video và binding (trùng từng byte với native).	Digest 32 byte, binding, b_{hi}/b_{lo} .
src/zk_proof.py, src/camera_proof.py	Payload pack/unpack, nén proof 129 byte, CameraProofBridge (snarkjs), build_payload/verify_payload.	Payload; quyết định verify fail-closed kèm lý do.
src/zk_setup.py	Setup Groth16: Powers of Tau công khai + Phase 2 có transcript.	camera_video.zkey, verification key.
src/native_blind_contract.py	Bản tham chiếu Python của HKDF/lịch/khung/whitening, segment_schedule.	Test vector chéo ngôn ngữ, lịch cho demo và digest.
src/api/app.py, src/api/native_handlers.py	Lỗi job dùng chung (xác thực, giới hạn, hàng đợi, lưu trữ) và dịch vụ native embed/extract/verify + WebSocket.	Job id, trạng thái, artifact, luồng live.
src/realtime_cavlc.py	Bộ lập lịch realtime có giới hạn theo đoạn.	Báo cáo p95, đoạn lỗi, accepted.
src/h264_tables.py	Bảng VLC CAVLC chuẩn cho phân giải thích của demo.	Giải mã độc lập để đối chiếu.
src/manifest.py, src/key_policy.py, src/trust/demo/	Manifest có chữ ký, chứng chỉ khóa có hạn, giao diện trust thử nghiệm.	Metadata trust nếu caller tích hợp.
	Demo duy nhất terminal_demo.py và các module deep_trace, h264_explain, segment_plan, negative_cases, service_steps, native_io.	Transcript, report.json, kiểm tra [PASS]/[FAIL].
src/runtest/	run_all.py (12 phase + H1), prepare_fixtures.py.	Mã thoát 0/1/2.
benchmark/	Một lệnh run_new_suite ghi một báo cáo duy nhất results/benchmark_report_new.pdf.	JSON/CSV kết quả và báo cáo PDF.

Nhận xét review. Hợp nhất về một lõi media (03/10/2026) vẫn là nền của kiến trúc: trước đó repository duy trì song song một pipeline Python (src/embedder.py, src/verifier*.py, src/core/, src/bitstream/) với parser, lịch và cách tái dựng khác native, cần file phụ hoặc video gốc khi xác minh. Sau khi gỡ, chỉ còn một parser, một lịch và một giao thức; mã cũ vẫn xem được trong lịch sử git. Đợt 04/10/2026 thêm một lệnh native (video-digest) và ba module Python nhỏ (sổ đăng ký, Poseidon, binding) thay vì một nhánh xử lý media thứ hai: hash video tái sử dụng chính parser và lịch native, và bản Python chỉ là tham chiếu để kiểm chéo.

4 Lớp zero-knowledge: proof camera ràng buộc video

4.1 Hai loại khóa và sổ đăng ký camera

Khóa	Ai giữ	Dùng để
Khóa giấu tin K (32 byte)	Người gửi và bên kiểm chứng	Chọn vị trí nhúng và làm trắng bit; xác định vị trí carrier khi tính digest video.
Secret camera s (phần tử trường BN254)	Chỉ camera	Tạo proof; số đăng ký chỉ lưu $pk = \text{Poseidon}(s)$.

Số đăng ký (`src/camera_registry.py`) là cây Merkle nhị phân độ sâu 16 (tối đa 65 536 camera), nút Poseidon(trái,phải), lá trống bằng 0, lá i là $pk_i = \text{Poseidon}(s_i)$. Poseidon cài bằng Python (`src/poseidon.py`, hàng số của circomlib) và đã được đối chiếu với vector chuẩn của circomlibjs và với witness của chính mạch. Bên kiểm chứng chỉ tin gốc (root) của cây.

4.2 Hash video và binding

Lệnh `zkstego_blind_bits video-digest` (bản tham chiếu `src/video_binding.py`) tính:

$$\begin{aligned} \text{frame_bits} &= 8(3 + |B|) \quad (\text{biết trước khi tạo proof}), \\ \text{carrier} &= \text{đúng frame_bits vị trí dấu mà lịch theo khóa } K \text{ sẽ ghi khung vào,} \\ d &= \text{SHA256}(\text{"video-digest/v1"} \parallel u_{32}(\text{frame_bits}) \parallel u_{32}(c) \parallel \parallel_{\text{NAL}} u_{32}(1 + |rb_{\text{sp}}'|) \parallel \text{header} \parallel rb_{\text{sp}}'), \end{aligned}$$

trong đó rb_{sp}' là RBSP của NAL (đã bỏ EPB) với *chỉ* các bit carrier được đặt về 0. Binding là

$$b = \text{SHA256}(\text{"proof-binding/v1"} \parallel \text{mode} \parallel d \parallel \text{SHA256}(m)),$$

tách thành hai nửa 128 bit b_{hi}, b_{lo} làm public input.

Nhận xét review. Nhúng chỉ đổi đúng các bit carrier (và có thể thêm/bớt byte EPB, không ảnh hưởng RBSP), nên digest của cover bằng digest của stego; phase 3 kiểm điều này và kiểm native trùng Python. Mọi thay đổi khác — video khác, cắt hoặc thêm frame, sửa một bit ở frame sau, lật một bit dấu không mang khung, mã hóa lại — làm digest đổi và proof không còn đúng. Lật một bit carrier giữ nguyên digest nhưng làm payload trích ra đổi, nên proof cũng bị từ chối. Đổi lại, vị trí carrier phụ thuộc K , nên chỉ người có khóa giấu tin mới tính lại được digest; bên kiểm chứng vốn đã cần K để trích payload, nên đây không phải yêu cầu mới, nhưng nghĩa là hệ thống chưa cho phép kiểm chứng bởi bên hoàn toàn không có khóa (mục 14).

Chế độ `mode = 1` chỉ ràng buộc message (digest bằng 32 byte 0) và dành cho luồng live, vì khi proof được nhúng thì phần còn lại của video chưa tồn tại. Job verify từ chối chế độ này (`video_binding_required`) trừ khi bật `ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1`.

4.3 Mạch camera_video.circom

Mạch có private input là secret s , `siblings[16]`, `pathIndices[16]` và ba public input `root`, `bindingHi`, `bindingLo`; tổng 8 737 constraint.

Listing 1: `circuits/camera_video.circom`, dòng 16–66

```

16 // Recomputes the root of a Poseidon Merkle tree from a Leaf and its path.
17 // pathIndices[i] = 0 when the running node is the left child at level i.
18 template MerkleInclusion(DEPTH) {
19     signal input leaf;
20     signal input siblings[DEPTH];
21     signal input pathIndices[DEPTH];
22     signal output root;
23
24     signal node[DEPTH + 1];
25     signal left[DEPTH];
26     signal right[DEPTH];
27     component hashes[DEPTH];
28     node[0] <== leaf;
29     for (var i = 0; i < DEPTH; i++) {
30         pathIndices[i] * (pathIndices[i] - 1) === 0;
31         left[i] <== node[i] + pathIndices[i] * (siblings[i] - node[i]);
32         right[i] <== siblings[i] + pathIndices[i] * (node[i] - siblings[i]);
33         hashes[i] = Poseidon(2);
34         hashes[i].inputs[0] <== left[i];
35         hashes[i].inputs[1] <== right[i];

```

```

36     node[i + 1] <== hashes[i].out;
37 }
38 root <== node[DEPTH];
39 }
40
41 template CameraVideo(DEPTH) {
42     signal input secret;
43     signal input siblings[DEPTH];
44     signal input pathIndices[DEPTH];
45     signal input root;
46     signal input bindingHi;
47     signal input bindingLo;
48
49     component publicKey = Poseidon(1);
50     publicKey.inputs[0] <== secret;
51
52     component membership = MerkleInclusion(DEPTH);
53     membership.leaf <== publicKey.out;
54     for (var i = 0; i < DEPTH; i++) {
55         membership.siblings[i] <== siblings[i];
56         membership.pathIndices[i] <== pathIndices[i];
57     }
58     membership.root == root;
59
60     // Public inputs that no other constraint uses are still part of the proof,
61     // but squaring them ties them into the R1CS explicitly (Semaphore-style).
62     signal bindingHiSquare <== bindingHi * bindingHi;
63     signal bindingLoSquare <== bindingLo * bindingLo;
64 }
65
66 component main {public [root, bindingHi, bindingLo]} = CameraVideo(16);

```

Nhận xét review. Dòng 30 ép mỗi `pathIndices[i]` là bit, nên prover không thể dùng giá trị trường tùy ý để trộn hai nút con; dòng 31–32 chọn thứ tự (trái, phải) bằng phép nội suy tuyến tính, dòng 58 buộc gốc tính được bằng `root` công khai. Hai public input binding không tham gia ràng buộc nào khác nên được bình phương (dòng 62–63) để chắc chắn chúng nằm trong R1CS. Proof tạo cho một binding (video + message) khác hoặc một root khác đều không verify được. Mạch SHA-256 cũ (`circuits/payload_verify.circom`, $c = \text{SHA256}(\text{SHA256}(m) \parallel \text{secret})$, 63 321 constraint, 513 public input) đã bị xóa; nó vừa bắt verifier giữ secret vừa không gắn với video. Đây là mô tả lịch sử, không còn là mã runtime.

4.4 Payload và nén proof

Payload là `[format 0x01][mode 1 byte][message_length 2 byte big-endian][message][proof nén 129 byte]`. Proof $A, C \in G_1$, $B \in G_2$ trên BN254 được nén thành tọa độ x (32 byte mỗi phần tử, B gồm hai phần tử $\mathbb{F}_p$) cộng một byte cờ chứa ba bit chẵn/lẻ của y . Khi giải nén, `bytes_to_proof` từ chối kích thước sai, bit cờ lạ, tọa độ không chính tắc ($\geq p$) và điểm không nằm trên đường cong (hoặc twist) bằng `ValueError`. Với khung native tối đa 4096 byte payload và 3 byte header khung, message tối đa $4096 - 4 - 129 = 3963$ byte.

Listing 2: `src/zk_proof.py`, dòng 195–216

```

195 →→
196 def pack_payload(mode: int, message: bytes, proof_bytes: bytes) -> bytes:
197     """[format 0x01][mode][message_len BE16][message][proof 129 B]``. """
198     if mode not in (MODE_VIDEO, MODE_MESSAGE_ONLY):
199         raise ValueError("binding mode must be 0 (video) or 1 (message only)")
200     if len(message) > MAX_MESSAGE_BYTES:
201         raise ValueError("message exceeds 65535 bytes")
202     if len(proof_bytes) != PROOF_SIZE_BYTES:
203         raise ValueError(f"proof_bytes must be {PROOF_SIZE_BYTES} bytes, got {len(proof_bytes)}")
204     return bytes((PAYLOAD_FORMAT, mode)) + len(message).to_bytes(2, "big") + message + proof_bytes
205
206
207 def unpack_payload(blob: bytes) -> CameraPayload:
208     """Strict inverse of :func:`pack_payload`; raises ValueError on any other layout."""
209     if len(blob) < PAYLOAD_HEADER_BYTES or blob[0] != PAYLOAD_FORMAT:
210         raise ValueError("payload format is not a camera proof payload")
211     if blob[1] not in (MODE_VIDEO, MODE_MESSAGE_ONLY):
212         raise ValueError("payload binding mode is invalid")
213     message_length = int.from_bytes(blob[2:4], "big")
214     if len(blob) != payload_size(message_length):
215         raise ValueError("payload length does not match its message length")
216     message = blob[PAYLOAD_HEADER_BYTES:PAYLOAD_HEADER_BYTES + message_length]
217     return CameraPayload(blob[1], message, blob[PAYLOAD_HEADER_BYTES + message_length:])

```

Nhận xét review. `unpack_payload` là nghịch đảo chặt của `pack_payload`: sai byte format, mode ngoài $\{0, 1\}$ hoặc tổng độ dài không đúng bằng $4 + |m| + 129$ đều là `ValueError`, và job verify ánh xạ thành `malformed_proof_payload` trước khi gọi `snarkjs`. Byte mode nằm trong binding, nên không thể hạ một proof video-bound xuống message-only mà vẫn verify.

4.5 Xác minh fail-closed

Listing 3: src/zk_proof.py, dòng 326–343

```

326 → def _snarkjs_verify(self, proof_dict: dict, public_signals: list) -> bool:
327     temp_dir = Path(tempfile.mkdtemp(prefix="zkp_verify_", dir=str(self.build_dir)))
328     proof_tmp, public_tmp = temp_dir / "proof.json", temp_dir / "public.json"
329     try:
330         proof_tmp.write_text(json.dumps(proof_dict), encoding="utf-8")
331         public_tmp.write_text(json.dumps(public_signals), encoding="utf-8")
332         result = self._run_snarkjs(["groth16", "verify", str(self.vkey_path), str(public_tmp), str(proof_tmp)])
333         # snarkjs exits 0 only for a valid proof and logs "OK!". Require both
334         # so a crash, missing file, or unexpected output never verifies.
335         stdout_lines = [line.strip() for line in (result.stdout or "").splitlines()]
336         reported_ok = any(line.endswith("OK!") for line in stdout_lines)
337         if result.returncode == 0 and reported_ok:
338             return True
339         if result.returncode == 0:
340             logger.error("[ZK] snarkjs exited 0 without reporting OK!; treating proof as invalid")
341         return False
342     finally:
343         shutil.rmtree(temp_dir, ignore_errors=True)

```

Listing 4: src/camera_proof.py, dòng 114–134

```

114 def check_proof(bridge: CameraProofBridge, root: int, unpacked: CameraPayload, proof: dict, *,
115                 native_cli: str | Path | None = None, stego: str | Path | None = None,
116                 stego_key: bytes | None = None, max_bits_per_idr: int = 64,
117                 require_video_binding: bool = True) -> Verdict:
118     """Rebuild the binding (from the stego video for mode 0) and verify the proof against ``root``."""
119     if unpacked.mode == MODE_MESSAGE_ONLY:
120         if require_video_binding:
121             return Verdict(False, "video_binding_required", unpacked.mode)
122         digest = bytes(DIGEST_BYTES)
123     else:
124         if native_cli is None or stego is None or stego_key is None:
125             raise ValueError("a video-bound payload needs the native CLI, the stego video and the stego key")
126         try:
127             digest = native_video_digest(native_cli, stego, stego_key, frame_bits_for_message(len(unpacked.message)),
128                                         max_bits_per_idr)
129         except ValueError:
130             return Verdict(False, "video_digest_unavailable", unpacked.mode)
131     binding = binding_digest(unpacked.mode, digest, unpacked.message)
132     if bridge.verify(proof, root, binding) is not True:
133         return Verdict(False, "proof_invalid", unpacked.mode)
134     return Verdict(True, None, unpacked.mode, unpacked.message)

```

Nhận xét review. Kết quả verify của snarkjs chỉ là True khi tiến trình vừa thoát mã 0 vừa in dòng kết thúc bằng OK!; crash, thiếu file hay output lạ đều thành False. check_proof tính lại digest từ *chính file nhận được*, dựng lại binding và verify với root tin cậy; is not True là so sánh chặt. Bên kiểm chứng cần root, verification key và khóa giấu tin K (để trích và định vị carrier), không cần secret camera, và không biết lá nào đã được dùng. Lỗi tính digest (video ngoài profile, hỏng) trả video_digest_unavailable thay vì bị hiểu là proof đúng.

4.6 Trusted setup

py -3.12 -m src.zk_setup: Phase 1 dùng Powers of Tau công khai của Hermez (2¹⁶, file powersOfTau28_hez_final_16.ptau với hash BLAKE2b được ghim); Phase 2 gồm 2 lượt đóng góp (entropy qua stdin), một beacon ngẫu nhiên sinh cục bộ, rồi snarkjs zkey verify; transcript ở circuits/build/camera_video_setup.json. Vì cả hai lượt chạy trên một máy nên đây là nghi thức demo; triển khai thật cần nhiều bên độc lập (chỉ cần một bên trung thực là khóa an toàn). Proof tạo bằng khóa của mạch cũ không verify với khóa hiện tại.

4.7 Proof chứng minh gì và không chứng minh gì

- **Chứng minh:** một camera có khóa trong sổ đăng ký đã xác nhận đúng video (mọi bit ngoài carrier trực tiếp, các bit carrier gián tiếp qua payload) và đúng message; không lộ camera nào; bên kiểm chứng không làm giả được proof vì không có secret camera.
- **Không chứng minh:** camera thật sự quay cảnh đó (ví dụ quay lại màn hình); tính bí mật của sự tồn tại payload (không có chứng minh không phát hiện được); nguồn gốc của luồng live chế độ mode = 1 ngoài message.

5 Kênh native CAVLC, giao thức v3

5.1 Ứng viên và lý do chọn dấu trailing-one

Lật `trailing_ones_sign_flag` không đổi độ dài mã, không đổi `TotalCoeff/TrailingOnes` nên không đổi ngữ cảnh `nC`, bảng VLC và cú pháp phía sau; bitstream vẫn hợp lệ. Native lấy một ứng viên mỗi khối residual (dấu trailing-one đầu tiên), thuộc mọi loại khối LumaDC, Luma4x4, ChromaDC, ChromaAC trong slice IDR. Định danh ứng viên có dạng `nal_index:mb:category:block:rbps_bit_offset`, tính theo đầu vào của đoạn (SPS+PPS+IDR), không theo vị trí trong file.

5.2 Tách khóa con bằng HKDF

Listing 5: `native/src/cavlc_stream.cpp`, dòng 1650–1713

```

1650 // ---- Blind channel protocol v3: HKDF key separation and whitening. The frame
1651 // carries no MAC: the payload's Groth16 proof is what a verifier checks.
1652
1653 constexpr std::size_t kSubkeyBytes = 32U;
1654 constexpr std::size_t kFrameHeaderBytes = 3U;
1655 constexpr std::size_t kKeystreamBlockBits = 256U;
1656 // Key-derivation labels are unchanged from v2, so v3 schedules and keystreams
1657 // equal the v2 ones for the same secret.
1658 constexpr std::string_view kHkdfSalt{"zkstego-cavlc-v2-salt"};
1659 constexpr std::string_view kScheduleInfo{"zkstego/cavlc/v2/schedule"};
1660 constexpr std::string_view kWhiteningInfo{"zkstego/cavlc/v2/whitening"};
1661
1662 std::string_view byte_view(const std::vector<std::uint8_t>& bytes) noexcept {
1663     return {reinterpret_cast<const char*>(bytes.data()), bytes.size()};
1664 }
1665
1666 // RFC 5869 HKDF-Extract. The returned PRK is secret: callers own and wipe it.
1667 std::vector<std::uint8_t> hkdf_sha256_extract(
1668     const std::vector<std::uint8_t>& salt,
1669     const std::vector<std::uint8_t>& input_key_material) {
1670     const std::vector<std::uint8_t> zero_salt(32U, 0U);
1671     HmacSha256 hmac(salt.empty() ? zero_salt : salt);
1672     auto digest = hmac.digest(byte_view(input_key_material));
1673     std::vector<std::uint8_t> prk(digest.begin(), digest.end());
1674     secure_wipe(digest.data(), digest.size());
1675     return prk;
1676 }
1677
1678 // RFC 5869 HKDF-Expand. Every intermediate block is wiped; the returned OKM
1679 // is owned (and wiped) by the caller.
1680 std::vector<std::uint8_t> hkdf_sha256_expand(
1681     const std::vector<std::uint8_t>& prk,
1682     const std::string_view info,
1683     const std::size_t length) {
1684     if (prk.size() < 32U) throw std::invalid_argument("HKDF pseudorandom key must be at least 32 bytes");
1685     if (length > 255U * 32U) throw std::invalid_argument("HKDF output length exceeds 255 SHA-256 blocks");
1686     HmacSha256 hmac(prk);
1687     std::vector<std::uint8_t> output;
1688     output.reserve(length + 32U);
1689     SensitiveBytes block_input(std::vector<std::uint8_t>{});
1690     block_input.value().reserve(32U + info.size() + 1U);
1691     std::array<std::uint8_t, 32> block{};
1692     for (std::size_t counter = 1U; output.size() < length; ++counter) {
1693         block_input.value().clear();
1694         if (counter > 1U) block_input.value().insert(block_input.value().end(), block.begin(), block.end());
1695         block_input.value().insert(block_input.value().end(), info.begin(), info.end());
1696         block_input.value().push_back(static_cast<std::uint8_t>(counter));
1697         block = hmac.digest(byte_view(block_input.value()));
1698         output.insert(output.end(), block.begin(), block.end());
1699     }
1700     secure_wipe(block.data(), block.size());
1701     secure_wipe(output.data() + length, output.size() - length);
1702     output.resize(length);
1703     return output;
1704 }
1705
1706 // One v2 subkey of a 32-byte channel secret. The caller wipes the result.
1707 std::vector<std::uint8_t> derive_cavlc_subkey(
1708     const std::vector<std::uint8_t>& secret_key,
1709     const std::string_view info) {
1710     if (secret_key.size() != 32U) throw std::invalid_argument("native CAVLC key must be 32 bytes");
1711     SensitiveBytes prk(hkdf_sha256_extract(
1712         std::vector<std::uint8_t>(kHkdfSalt.begin(), kHkdfSalt.end()), secret_key));
1713     return hkdf_sha256_expand(prk.value(), info, kSubkeyBytes);

```

Nhận xét review. Từ giao thức v3 (04/10/2026) chỉ còn hai khóa con sinh bằng HKDF-SHA256 (RFC 5869) từ khóa giấu tin 32 byte: PRK = HMAC(“zkstego-cavlc-v2-salt”, K), schedule_key và whitening_key với info zkstego/cavlc/v2/{schedule, whitening}. Nhân giữ nguyên như v2 nên lịch và dòng làm trắng không đổi với cùng khóa; khóa con thứ ba của v2 (dùng cho thẻ HMAC của khung) đã bị bỏ cùng với thẻ. Lộ một khóa con không làm lộ K hay khóa con còn lại. Implementation xóa PRK và các khối trung gian (SensitiveBytes, secure_wipe). Python (src/native_blind_contract.py) cài cùng ABI và phase 2 kiểm test vector chéo ngôn ngữ. HMAC ở đây chỉ đóng vai hàm giả ngẫu nhiên có khóa, không còn dùng để xác thực.

5.3 Lịch, khung và whitening

Lịch: score = HMAC-SHA256(k_s , định danh ASCII); sắp ứng viên tăng dần theo (score, định danh), lấy $N = \min(\max_bits_per_IDR, \text{số ứng viên của đoạn})$ (mặc định 64 bit/IDR). Khung: $[0 \times 03][payload_len \text{ 2 byte big-endian}][payload]$, tức 3 byte overhead và không có MAC. Bit nhúng thứ i = bit khung thứ i XOR bit keystream thứ i , với keystream = HMAC(k_w , uint64_be(0)) || HMAC(k_w , uint64_be(1)) || ...

Listing 6: native/src/cavlc_stream.cpp, dòng 1740–1792

```

1740 // Keystream block j = HMAC-SHA256(whitening_key, uint64_be(j)).
1741 std::array<std::uint8_t, 32> whitening_block(HmacSha256& hmac, const std::uint64_t block_index) {
1742     std::array<char, 8> counter{};
1743     for (std::size_t index = 0; index < counter.size(); ++index) {
1744         counter[index] = static_cast<char>((block_index >> (56U - 8U * index)) & 0xffU);
1745     }
1746     return hmac.digest(std::string_view(counter.data(), counter.size()));
1747 }
1748
1749 // Returns bits[i] XOR keystream bit (first_bit_index + i). The bit index is
1750 // global to one frame, so a stream session whitens each segment by passing
1751 // the number of frame bits already carried by earlier segments. XOR is an
1752 // involution: the same call un-whitens extracted bits.
1753 std::vector<std::uint8_t> whiten_cavlc_frame_bits(
1754     const std::vector<std::uint8_t>& bits,
1755     const std::vector<std::uint8_t>& whitening_key,
1756     const std::size_t first_bit_index) {
1757     std::vector<std::uint8_t> output;
1758     output.reserve(bits.size());
1759     if (bits.empty()) return output;
1760     HmacSha256 hmac(whitening_key);
1761     std::array<std::uint8_t, 32> block{};
1762     std::uint64_t loaded_block = 0;
1763     bool block_loaded = false;
1764     for (std::size_t index = 0; index < bits.size(); ++index) {
1765         if (bits[index] > 1U) throw std::invalid_argument("CAVLC frame bit is invalid");
1766         const auto global_bit = static_cast<std::uint64_t>(first_bit_index + index);
1767         const auto block_index = global_bit / kKeystreamBlockBits;
1768         if (!block_loaded || block_index != loaded_block) {
1769             block = whitening_block(hmac, block_index);
1770             loaded_block = block_index;
1771             block_loaded = true;
1772         }
1773         const auto bit_in_block = static_cast<std::size_t>(global_bit % kKeystreamBlockBits);
1774         const auto key_bit = static_cast<std::uint8_t>((block[bit_in_block / 8U] >> (7U - bit_in_block % 8U)) & 1U);
1775         output.push_back(static_cast<std::uint8_t>(bits[index] ^ key_bit));
1776     }
1777     secure_wipe(block.data(), block.size());
1778     return output;
1779 }
1780
1781 std::vector<std::uint8_t> pack_frame(const std::vector<std::uint8_t>& payload) {
1782     if (payload.size() > std::numeric_limits<std::uint16_t>::max()) {
1783         throw std::invalid_argument("CAVLC payload exceeds 65535 bytes");
1784     }
1785     std::vector<std::uint8_t> frame;
1786     frame.reserve(kFrameHeaderBytes + payload.size());
1787     frame.push_back(kCavlcFrameVersion);
1788     frame.push_back(static_cast<std::uint8_t>(payload.size() >> 8U));
1789     frame.push_back(static_cast<std::uint8_t>(payload.size()));
1790     frame.insert(frame.end(), payload.begin(), payload.end());
1791     return frame;
1792 }

```

Nhận xét review. Không có whitening thì byte version cố định, trường độ dài nhỏ và cấu trúc proof làm thống kê dấu tại các vị trí được chọn bị lệch; benchmark 04/10/2026 đo $|z| \leq 0,22$ cho tỉ lệ dấu âm trước/sau nhúng. Chỉ số bit keystream là toàn cục trong một khung (`first_bit_index`), nên khung trải qua nhiều IDR vẫn dùng một dòng khóa liên tục; XOR là phép tự nghịch đảo nên cùng hàm dùng cho bỏ whitening. Keystream chỉ phụ thuộc K và chỉ số bit, không có nonce: dùng cùng K cho nhiều video thì dòng khóa lặp lại (mục 14). `pack_frame` (dòng 1781–1792) cho thấy khung v3 chỉ còn version, độ dài và payload.

5.4 Một giao thức theo đoạn và trích xuất mù

Chỉ có một giao thức, theo đoạn (SPS+PPS+1 IDR), dùng chung cho file, job HTTP và luồng live. Mỗi đoạn mang tối đa `max_bits_per_IDR` bit; khung trải qua nhiều IDR liên tiếp và chỉ số bit khung đếm liên tục qua các đoạn. Bên nhận không cần video gốc hay file phụ: parse lại IDR, sinh lại khóa con, tái tạo lịch, bỏ whitening 24 bit đầu để kiểm version = 3 và độ dài $\leq$ giới hạn, rồi đọc đủ khung. Sai khóa thì byte version sau khi bỏ whitening gần như ngẫu nhiên nên bị từ chối ngay; lọt qua cả version lẫn độ dài với xác suất cỡ $\frac{1}{256} \cdot \frac{\text{giới hạn}}{65536}$, và phần còn sót bị loại khi giải nén proof hoặc ở Groth16.

Nhận xét review. Thông báo lỗi native được phân loại rõ: version sai, độ dài vượt giới hạn hoặc không khớp, lịch thiếu dung lượng, hết video trước khi đọc đủ khung; mọi trường hợp đều thoát mã 2. Vì khung không còn MAC, tìm thấy khung chỉ có nghĩa là có dữ liệu ở đúng vị trí theo khóa; payload trả về từ lệnh/job extract và luồng live mang `"verified": false`, và *chỉ* job verify (Groth16) xác thực nó. Lý do bỏ thẻ HMAC 16 byte của v2: khi Groth16 verify đã bắt buộc, thẻ trùng việc (sửa message thì proof sai, sửa proof thì proof không hợp lệ), còn bỏ thẻ tiết kiệm 128 bit, tức 2 IDR ở 64 bit/IDR. v3 không đọc được stego v2/v1 (byte version khác); đây là thay đổi có chủ đích. `zkstego_inspect --segments` và `segment_schedule` cho phép caller tái tạo đúng lịch của encoder, đây là cơ sở để demo và hash video định vị từng bit.

6 Native CLI và các sửa lỗi hardening

6.1 Lệnh và ranh giới tiến trình

`zkstego_blind_bits` có các nhóm lệnh:

- `embed-stream-auth-stdin`, `extract-stream-auth`: khung v3, lịch theo đoạn IDR, đọc/ghi file; hậu tố `-auth` giữ lại vì tương thích tên lệnh và nay chỉ có nghĩa “có khóa”;
- `video-digest`: hash ràng buộc video; cover và stego cho cùng giá trị;
- `measure-live-capacity-stdin`: in JSON dung lượng ứng viên;
- `embed-live-auth-stdin`, `extract-live-auth-stdin`: video vào/ra qua stdin/stdout, dùng cho WebSocket.

Khóa luôn là một dòng 64 ký tự hex trên stdin, không bao giờ qua argv. Các công cụ soi trước đây được gộp vào một công cụ duy nhất `zkstego_inspect`.

6.2 Bộ đọc Annex-B từ stdin

Listing 7: `native/src/cavlc_stream.cpp`, dòng 1237–1282

```
1237 void AnnexBNalStreamReader::compact() {
1238     if (head_ == 0U) return;
1239     buffer_.erase(buffer_.begin(), buffer_.begin() + static_cast<std::ptrdiff_t>(head_));
1240     scan_ = scan_ > head_ ? scan_ - head_ : 0U;
1241     head_ = 0U;
1242 }
1243
1244 void AnnexBNalStreamReader::read_more() {
1245     if (eof_) return;
1246     // Drop already-returned bytes at most once per returned NAL.
1247     compact();
1248     auto* source = input_.rdbuf();
1249     if (source == nullptr) throw std::invalid_argument("Annex-B input stream is unavailable");
1250     constexpr std::streamsize chunk_size = 64 * 1024;
1251     // Only request what the stream buffer promises it can deliver without
1252     // blocking (in_avail), so a live pipe never waits to fill a whole chunk.
1253     const auto append_available = [&](const std::streamsize limit) {
1254         const auto available = source->in_avail();
1255         if (available <= 0 || limit <= 0) return;
1256         const auto requested = std::min(available, limit);
1257         const auto old_size = buffer_.size();
1258         buffer_.resize(old_size + static_cast<std::size_t>(requested));
```

```

1259     const auto received = source->sgetn(
1260         reinterpret_cast<char*>(buffer_.data() + old_size), requested);
1261     // in_avail() promised `requested` bytes; anything else is a stream
1262     // fault, and accepting zero here could spin forever.
1263     if (received <= 0 || received > requested) {
1264         buffer_.resize(old_size);
1265         throw std::invalid_argument("failed while reading Annex-B input stream");
1266     }
1267     buffer_.resize(old_size + static_cast<std::size_t>(received));
1268 };
1269 if (source->in_avail() > 0) {
1270     append_available(chunk_size);
1271     return;
1272 }
1273 // Nothing buffered: block for a single byte, then drain anything that
1274 // arrived with it.
1275 const auto next = source->sbump();
1276 if (std::char_traits<char>::eq_int_type(next, std::char_traits<char>::eof())) {
1277     eof_ = true;
1278     return;
1279 }
1280 buffer_.push_back(static_cast<std::uint8_t>(std::char_traits<char>::to_char_type(next)));
1281 append_available(chunk_size - 1);
1282 }

```

Nhận xét review. Đợt 02/10/2026 sửa một bộ đọc stdin có chi phí bậc hai: byte đã trả về bị xóa khỏi đầu buffer và NAL đang chờ bị quét lại từ đầu sau mỗi lần đọc. Hiện `compact()` chỉ dồn buffer tối đa một lần cho mỗi NAL trả về, việc quét start code tiếp tục từ `scan_` thay vì quét lại, và `read_more()` chỉ lấy lượng `in_avail()` hứa có sẵn nên pipe live không bị chặn chờ đẩy chunk. Cùng với tăng tốc bộ đọc bit, một lần live embed 1080p all-intra giảm từ 30,1 s xuống 2,5 s trên máy phát triển (số liệu lịch sử, đo ngày 02/10/2026 với giao thức v2; đo trên host, không phải nghiệm thu realtime).

6.3 Giới hạn tài nguyên và đường dẫn

- **Kích thước ảnh từ SPS:** tối đa 36 864 macroblock (MaxFS của level 5.1/5.2, bao phủ 3840×2160) và tối đa 543 MB mỗi chiều; SPS vượt giới hạn bị từ chối thay vì để một SPS độc hại phóng to bộ nhớ decoder.
- **Kích thước đầu vào:** file Annex-B đọc toàn bộ bị giới hạn 1 GiB; `zkstego_inspect` giới hạn 64 MiB; NAL trong luồng bị giới hạn theo cấu hình.
- **Đường dẫn Unicode và dài trên Windows:** hai công cụ vào qua `wmain`, chuyển tham số sang UTF-8 rồi mở file qua `std::filesystem::path/_wopen_s` với tiền tố `\\?\\`, nên repository và video có tên tiếng Việt hoặc đường dẫn dài vẫn chạy.

7 Dịch vụ HTTP/WebSocket

7.1 Xác thực trước khi đọc body

Listing 8: `src/api/app.py`, dòng 181–224

```

181 async def __call__(self, scope, receive, send):
182     if scope["type"] != "http":
183         await self.app(scope, receive, send)
184         return
185     # Keep the first occurrence of each header, matching FastAPI's Header() dependency.
186     headers: dict[bytes, bytes] = {}
187     for key, value in scope.get("headers", ()):
188         headers.setdefault(key.lower(), value)
189     # Compare against the routed path: with --root-path, uvicorn prefixes scope["path"],
190     # and a raw prefix check would let protected routes skip auth and size limits.
191     path = get_route_path(scope) if scope.get("path") else ""
192     if self.api_token is not None and path.startswith(self._PROTECTED_PREFIX):
193         client = client_host(scope)
194         retry_after = self.auth_limiter.retry_after(client) if self.auth_limiter is not None else 0
195         if retry_after:
196             await self._reject(
197                 send, 429, b'{"detail": "too many failed authentication attempts; retry later"}',
198                 extra_headers=[(b"retry-after", str(retry_after).encode("ascii"))],
199             )
200         return
201     if not bearer_token_matches(headers.get(b"authorization"), self.api_token):
202         failures = self.auth_limiter.record_failure(client) if self.auth_limiter is not None else None
203         _LOGGER.warning(
204             "API authentication failed: client=%s path=%r failures_in_window=%s", client, path, failures,
205         )
206         await self._reject(
207             send, 401, b'{"detail": "Bearer token required"}',

```

```

208         extra_headers=[(b"www-authenticate", b"Bearer")],
209     )
210     return
211 if scope.get("method") != "POST" or path not in self._UPLOAD_PATHS:
212     await self.app(scope, receive, send)
213     return
214 if self.max_concurrent_uploads is not None and self._active_uploads >= self.max_concurrent_uploads:
215     await self._reject(
216         send, 503, b'{"detail":"upload capacity reached; retry later"}',
217         extra_headers=[(b"retry-after", b"1")],
218     )
219     return
220 self._active_uploads += 1
221 try:
222     await self._guard_upload(scope, receive, send, headers)
223 finally:
224     self._active_uploads -= 1

```

Nhận xét review. FastAPI phân tích (và spool) multipart trước khi dependency của route chạy, nên kiểm token ở dependency nghĩa là server đã nhận body của client chưa xác thực. Middleware `RequestBodyGuard` kiểm Bearer token trước khi đọc bất kỳ byte body nào. Dòng 189–191 sửa một lỗi bypass: với `--root-path`, Uvicorn thêm tiền tố vào `scope["path"]`, nên so tiền tố thô có thể để route được bảo vệ bỏ qua xác thực và giới hạn kích thước; hiện guard so trên routed path. Client sai token 10 lần trong 60 s nhận 429 kèm `Retry-After` (cả HTTP lẫn WebSocket). Token phải dài tối thiểu 32 ký tự, kiểm cả khi khởi động lần trong `create_app`.

7.2 Job verify với proof camera bắt buộc

Listing 9: `src/api/native_handlers.py`, dòng 56–88

```

56 def make_proof_verify_handler(
57     extract_payload: PayloadExtractor, check: ProofChecker,
58 ) -> Callable[..., dict[str, Any]]:
59     """Blind-extract a camera payload and accept it only if its Groth16 proof verifies.
60
61     The stego key only extracts; the proof is checked against the trusted camera
62     registry root and, for video-bound payloads, the digest of the uploaded video.
63     Returns ``{"valid": True, "message_b64", "payload_bytes", "video_bound"}`` for a
64     verified proof and ``{"valid": False, "reason"}`` otherwise (payload_not_found,
65     malformed_proof_payload, video_binding_required, video_digest_unavailable,
66     proof_invalid). Infrastructure errors (timeouts, missing Node.js/circuits/registry)
67     propagate, so the job service marks the job "failed" rather than "rejected".
68     """
69
70     def verify_handler(*, stego_video_path: str, secret_key: bytes, maximum_payload_bytes: int) -> dict[str, Any]:
71         try:
72             blob = extract_payload(stego_video_path, secret_key, maximum_payload_bytes)
73         except NativePayloadNotFound:
74             return {"valid": False, "reason": "payload_not_found"}
75         parsed = parse_payload(blob)
76         if parsed is None:
77             return {"valid": False, "reason": "malformed_proof_payload"}
78         verdict = check(*parsed, stego_video_path, secret_key)
79         if not verdict.valid:
80             return {"valid": False, "reason": verdict.reason or "proof_invalid"}
81         return {
82             "valid": True,
83             "message_b64": base64.b64encode(verdict.message or b"").decode("ascii"),
84             "payload_bytes": len(blob),
85             "video_bound": verdict.video_bound,
86         }
87
88     return verify_handler

```

Listing 10: `src/api/native_handlers.py`, dòng 198–208

```

198 def camera_check(payload: CameraPayload, proof: dict, stego_video_path: str, secret_key: bytes) -> Verdict:
199     # A missing registry or verification key is a deployment fault ("failed"), not an invalid proof.
200     if settings.camera_registry_path is None:
201         raise RuntimeError("camera registry is not configured")
202     registry = CameraRegistry.load(settings.camera_registry_path)
203     bridge = CameraProofBridge(settings.circuits_dir)
204     if not bridge.vkey_path.is_file():
205         raise RuntimeError("Groth16 verification key is not available")
206     return check_proof(bridge, registry.root, payload, proof, native_cli=executable, stego=stego_video_path,
207                        stego_key=secret_key, max_bits_per_idr=max_bits_per_idr,
208                        require_video_binding=settings.require_video_binding)

```

Nhận xét review. POST /api/v1/jobs/verify trích mù, unpack, giải nén điểm, tính digest video của chính file tải lên, rồi bắt buộc Groth16 với root của sổ đăng ký (ZK_STEGO_CAMERA_REGISTRY). Kết quả hợp lệ duy nhất là succeeded kèm message và video_bound; mọi trường hợp khác là rejected với lý do payload_not_found, malformed_proof_payload, video_binding_required, video_digest_unavailable hoặc proof_invalid. Thiếu sổ đăng ký, thiếu verification key, timeout hay thiếu Node.js là lỗi hạ tầng (dòng 200–205 ném RuntimeError) nên job thành failed, không bị nhầm thành “proof sai”. Khóa giấu tin của request chỉ dùng để trích và định vị carrier; không có secret camera nào ở phía dịch vụ.

7.3 Các bảo vệ khác

Cơ chế	Hiện trạng
Trạng thái job	queued, running, succeeded, rejected, failed; rejected là trạng thái kết thúc và client không được coi là thành công.
Payload chưa xác thực	Kết quả job extract và payload của luồng live mang "verified": false; chỉ job verify xác thực.
Sổ đăng ký và chế độ proof	ZK_STEGO_CAMERA_REGISTRY bắt buộc cho job verify; proof mode = 1 bị từ chối trừ khi ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1.
Upload	Tối đa 4 upload đồng thời; giới hạn kích thước; hạn thời gian upload ZK_STEGO_API_UPLOAD_TIMEOUT_SECONDS (mặc định 120 s).
Hàng đợi	Có giới hạn (mặc định 1 worker + 2 chờ); đầy thì 503 kèm Retry-After.
Lưu trữ	Artifact tải một lần, hết hạn sau 10 phút; job đã xong bị xóa sau ZK_STEGO_API_JOB_RETENTION_SECONDS (mặc định 24 h).
WebSocket	Từ chối trước khi accept nếu thiếu token; đóng sau ZK_STEGO_STREAM_IDLE_TIMEOUT_SECONDS (15 s) không có dữ liệu.
Tiến trình native	Không qua shell, khóa qua stdin, có timeout, stderr được đọc liên tục.
OpenAPI	/docs chỉ bật khi ZK_STEGO_API_DOCS=1.

8 Bộ lập lịch realtime

src/realtime_cavlc.py (RealtimeCAVLCScheduler) xử lý từng đoạn IDR với hàng đợi giới hạn theo số đoạn và số byte (bỏ đoạn cũ nhất khi đầy), epoch tăng đơn điệu và cache proof tối đa 4 epoch. Báo cáo gồm số đoạn bị bỏ, số đoạn lỗi, p95 thời gian patch, p95 độ trễ đầu–cuối và p95 thời gian tạo proof; accepted chỉ đúng khi không có đoạn lỗi và p95 nằm trong ngân sách một frame.

Nhận xét review. Công nghiệm thu được thiết kế fail-closed, nhưng kết quả đo camera thật hiện chưa vượt công 30 FPS (mục 14). Bộ lập lịch là bằng chứng chức năng về giới hạn bộ nhớ và epoch, không phải bằng chứng hiệu năng. Luồng live chỉ ràng buộc được message (mode = 1), vì proof được nhúng trước khi phần còn lại của video tồn tại.

9 Demo và kiểm thử

9.1 Một demo duy nhất

Bộ script demo theo từng bước trước đây đã được gộp vào demo/terminal_demo.py ngày 03/10/2026. Demo dùng cùng giao thức theo đoạn với dịch vụ, và phần deep trace đối chiếu mọi giá trị tự tính với FFmpeg hoặc parser native: pipeline FFmpeg/libx264 từ log verbose; cắt NAL, EBSP→RBSP; toàn bộ SPS/PPS/slice header qua trace_headers; bản đồ loại MB và QP từ decoder FFmpeg; vòng đời đầy đủ của một khối 4×4 (bit header → phần tử CAVLC → hệ số → giải lượng tử → biến đổi ngược → dự đoán intra → pixel, khớp từng mẫu với ảnh FFmpeg -skip_loop_filter all); vị trí bit nhúng trong file và trường payload mà bit đó mang. Phần lịch hiển thị dẫn xuất HKDF, khung v3 và phép XOR với keystream. Sau đó demo chạy các ca từ chối (đổi message, proof sai, thiếu proof, sai khóa, lật một bit mang dữ liệu), job HTTP thật (embed trùng từng byte với CLI, extract, verify → succeeded; verify sai khóa → rejected) và luồng WebSocket. Mọi kiểm tra in [PASS]/[FAIL]; một [FAIL] dừng demo với mã thoát 1.

9.2 Bộ kiểm thử

Ngày 04/10/2026, src/runtest/run_all.py qua **142/142 trên 12 phase** trên host Windows với fixture từ prepare_fixtures.py; CTest native qua 1/1. Phase camera thật H1 (--hardware) không nằm trong lần chạy này. Mã thoát run_all: 0 = tất cả qua, 1 = có lỗi, 2 = có ca bắt buộc bị bỏ qua.

Phase	File	Nội dung
1	test_zk_proof.py	Định dạng proof, payload, vector Poseidon, sổ đăng ký, binding, kiểm chéo mạch, Groth16 prove/verify thật.
2	test_native_blind_contract.py	C++ và Python khớp HKDF, lịch, khung, whitening, lịch theo đoạn.
3	test_native_cli_fixture.py	CLI nhúng/trích trên fixture 300 frame, decode nghiêm ngặt, sai khóa, digest video (native = Python, cover = stego).
4	test_h264_tables.py	Bảng VLC CAVLC (prefix-free, Kraft, giá trị chuẩn).
5	test_service_api.py	Lỗi job, xác thực trước body, giới hạn thử sai, lưu trữ, job verify, hạn khóa.
6	test_native_http_channel.py	Job HTTP qua Uvicorn thật (proof camera ràng buộc video được chấp nhận; payload chép sang video khác và video bị cắt bị từ chối) và luồng WebSocket.
7	test_manifest_security.py	Ký manifest, ràng buộc positions/stego.
8	test_realtime_scheduler.py	Bộ lập lịch realtime.
9	test_benchmark_recorder.py	Kiểm tra fail-closed của bộ ghi benchmark camera và các hàm phân tích benchmark.
10	test_trust_interfaces.py	Giao diện trust thử nghiệm (provenance, C2PA, attestation).
11	test_demo_h264_explain.py	Toán giải thích của demo khớp native và pixel FFmpeg.
12	test_demo_smoke.py	Toàn bộ demo chạy không có [FAIL].
H1	test_native_camera_http.py	Camera thật (--hardware).

10 Kết quả benchmark đã ghi (04/10/2026)

Lệnh py -3.12 -m benchmark.run_new_suite chạy mọi phép đo và ghi một báo cáo benchmark/results/benchmark_report_new.pdf. Lần chạy được trích dẫn: 04/10/2026, i7-12700H, Windows 11, giao thức v3 + proof camera; media run 20261004T142448Z_1db00a, e2e_new.json, zkp_new.json. Ma trận media gồm 9 clip CIF (akiyo, city, coastguard, container, deadline, football, foreman, foreman_cif_300f, hall_monitor) × 3 độ phân giải (352×288, 640×480, 1280×960; hai mức lớn được scale từ CIF), libx264 Baseline/CAVLC, QP 22, all-intra, 30 frame đầu; mỗi case dùng khóa mới và một camera của sổ đăng ký 8 camera.

Hạng mục	Kết quả
Chức năng	Chuyển đổi 9/9, media 27/27 (mọi proof ràng buộc video), E2E 5/5, tần công 11/11 đúng kỳ vọng; 0 dấu bị đổi ngoài lịch.
Payload	201 byte (header 4 + message 68 + proof 129) → khung 204 byte = 1 632 bit → 26 IDR ở 64 bit/IDR; khoảng một nửa số dấu trong lịch thực sự bị lật.
E2E (median, clip CIF 300 frame)	Digest cover 0,51 s; Groth16 prove 1,77 s; embed 0,50 s; decode nghiêm ngặt 0,15 s; extract 0,53 s; verify (digest stego + Groth16) 1,45 s; tổng gửi 2,78 s, tổng nhận 1,99 s. Clip có 300 đoạn IDR, 188 556 dấu ứng viên, dung lượng 19 200 bit ở 64 bit/IDR.
Hệ proof (cùng R1CS 8737 constraint)	Groth16 prove 1,5–1,8 s, verify 0,9–1,0 s, proof 129 byte; PLONK prove khoảng 22 s (21,5–23,3 s), verify 0,90–0,94 s, proof khoảng 2,25 KB JSON, setup 3,2 s. Mạch SHA-256 cũ cần 3,5–4,4 s để prove (số liệu lịch sử để so sánh).
Chi phí native	Median quét dung lượng mỗi IDR 52 / 220 / 770 ms; embed mỗi IDR mang dữ liệu 61 / 245 / 891 ms; digest video (parse 26 IDR carrier) 1,6 / 6,4 / 22,6 s (lần lượt 352×288 / 640×480 / 1280×960). Parse, không phải Groth16, giới hạn thông lượng.

Hạng mục	Kết quả
Chất lượng	PSNR-Y cover→stego thấp nhất mỗi case 44,03–58,68 dB, 0/27 dưới 40 dB; 26/30 frame thay đổi ở mọi case; kích thước stego bằng cover.
Thống kê dấu	$ z \leq 0,22$ cho tỉ lệ dấu âm của trailing one.

Ma trận tấn công (quyết định của job verify HTTP, e2e_new.json):

Ca	Thao tác	Kết quả
baseline	Stego gốc, đúng khóa, đúng số đăng ký	chấp nhận
wrong_key	Khóa giấu tin sai	payload_not_found
flip_scheduled_sign	Lật 1 dấu nằm trong lịch (giữa khung)	malformed_proof_payload
flip_unscheduled_carrier_sign	Lật 1 dấu ứng viên ngoài lịch, trong IDR mang khung	proof_invalid
flip_later_frame_sign	Lật 1 dấu ở IDR cuối (ngoài vùng nhúng)	proof_invalid
drop_first_idr	Xóa IDR mang bit đầu tiên của khung	payload_not_found
truncate_half	Cắt còn 50% đầu stream (payload còn nguyên)	proof_invalid
transcode	Giải mã rồi mã hóa lại (libx264, QP 22)	payload_not_found
message_swap	Người có khóa giấu tin thay message, giữ proof cũ	proof_invalid
unregistered_camera	Camera ngoài số đăng ký tự tạo proof với cây của nó	proof_invalid
replay_other_video	Chép payload sang video khác (akiyo) cùng khóa	proof_invalid

Nhận xét review. Các kết quả media/camera trước 04/10/2026 (giao thức v1/v2, mạch SHA-256) không còn là bằng chứng hiện tại và đã được thay bằng lần chạy trên. Ngoại lệ là các lần đo camera thật ngày 24/09/2026: chúng chưa được đo lại và chỉ được trích như số liệu lịch sử (mục 14). Benchmark không đo steganalysis bằng học máy, so sánh với phương pháp giấu tin video khác, thiết bị edge, CABAC hay carrier P/B-frame.

11 Điểm mạnh của hệ thống hiện tại

- **Một lõi media, một giao thức.** Một parser native, một lịch theo đoạn dùng chung cho file, HTTP, live và hash video; không còn hai implementation phải giữ đồng bộ.
- **Nhúng không phá cú pháp.** Kênh dấu trailing-one giữ nguyên độ dài mã và ngữ cảnh CAVLC; output được kiểm bằng decode FFmpeg nghiêm ngặt; kích thước stego bằng cover; PSNR-Y thấp nhất 44,03 dB trên 27 case.
- **Proof gắn với video, message và số đăng ký.** Phát lại payload sang video khác, cắt video, sửa frame sau, lật dấu ngoài carrier, thay message hay dùng camera ngoài số đăng ký đều bị Groth16 từ chối (11/11 ca tấn công đúng kỳ vọng).
- **Verifier không cần secret camera.** Bên kiểm chứng chỉ cần root, verification key và khóa giấu tin; không làm giả được proof và không biết camera nào đã ký.
- **Tách khóa và làm trắng.** HKDF tách vai trò lịch/whitening; whitening loại bỏ cấu trúc cố định của khung khỏi thống kê dấu ($|z| \leq 0,22$).
- **Trích xuất mù gọn.** Không cần video gốc hay sidecar; khung chỉ tốn 3 byte overhead; sai khóa bị loại rẻ ở kiểm version/độ dài.
- **Xác minh fail-closed.** Kiểm điểm đường cong khi giải nén; snarkjs phải vừa thoát 0 vừa in OK!; job verify chỉ succeeded khi proof đúng; lỗi hạ tầng là failed, không phải rejected.
- **Ranh giới dịch vụ được làm cứng.** Xác thực trước body, throttling, giới hạn kích thước/thời gian/hàng đợi, lưu trữ có hạn, khóa không qua argv hay log.
- **Có thể kiểm chứng từng bit.** zkstego_inspect, segment_schedule, bản tham chiếu Python của digest và deep trace của demo cho phép đối chiếu mọi bước với FFmpeg và parser native; test chéo ngôn ngữ chặn lệch ABI.

12 Các vấn đề đã xác minh và sửa trong các đợt review 02–04/10/2026

Vấn đề	Rủi ro	Cách sửa hiện tại
Proof không gắn với video hay nội dung đoạn (bản 03/10)	Payload phát lại sang video khác dùng cùng khóa vẫn được chấp nhận; cắt video không bị phát hiện	Đã sửa 04/10: binding chứa digest toàn video (trừ bit carrier) và hash message; ca replay_other_video, truncate_half, flip_later_frame_sign đều proof_invalid.
Verifier phải biết secret để tính lại commitment (bản 03/10)	Không kiểm chứng công khai được; ai giữ khóa cũng tạo được proof	Đã sửa 04/10: secret camera s chỉ ở camera, tách khỏi khóa giấu tin K; verifier cần root + verification key + K; ca message_swap bị từ chối dù kẻ tấn công có K.
Không có danh tính nguồn	Không biết payload do thiết bị được phép tạo	Số đăng ký Merkle Poseidon; camera ngoài sổ (unregistered_camera) bị từ chối; không lộ camera nào.
Mạch SHA-256 cũ không ép bit (lịch sử)	Soundness: prover đưa giá trị trường tùy ý vào Sha256	Đã sửa 03/10 bằng ràng buộc bit, rồi thay hẳn mạch bằng camera_video ngày 04/10 (ép pathIndices là bit).
Trusted setup cục bộ không có nguồn công khai	Không truy được nguồn tham số	Phase 1 dùng Powers of Tau công khai Hermez (hash ghim), Phase 2 có transcript và zkey verify (vẫn là nghi thức demo).
Verify dựa vào mã thoát snarkjs	Crash/output lạ có thể bị hiểu nhầm	Yêu cầu exit 0 và OK!; bytes_to_proof kiểm tọa độ chính tắc và điểm trên đường cong.
Một khóa cho nhiều vai trò; khung không làm trắng (trước 03/10)	Lộ một vai trò ảnh hưởng vai trò khác; thống kê dấu lệch	HKDF tách khóa con lịch/whitening; whitening bằng keystream HMAC (v2, giữ nguyên ở v3).
Thẻ HMAC của khung v2 trùng việc với Groth16	16 byte overhead (128 bit, khoảng 2 IDR) không thêm bảo đảm khi Groth16 đã bắt buộc	Giao thức v3 (04/10): khung [0x03][len][payload], payload trích ra mang "verified": false cho tới khi job verify kiểm proof.
Nhiều chế độ nhúng cho file và live	Lịch khác nhau giữa các đường vào	Một giao thức theo đoạn IDR cho file, HTTP và live.
Hai lỗi media (native và Python)	Hai implementation lệch nhau, Python cần sidecar/video gốc	Gỡ pipeline Python và benchmark SEC (khoảng 18 000 dòng); một lỗi native.
Benchmark đã ghi dùng giao thức v1 và mạch cũ	Số liệu không áp dụng cho phiên bản hiện tại	Chạy lại 04/10 với v3 + proof camera: media 27/27, 0/27 dưới 40 dB, E2E 5/5, tấn công 11/11.
Nhiều công cụ soi ròi rác	Kết quả soi không nhất quán	Một công cụ zkstego_inspect.
Token kiểm sau khi parse multipart	Client chưa xác thực đầy đủ body lên server	RequestBodyGuard xác thực trước khi đọc body.
Kiểm tiền tố thô khi chạy sau --root-path	Bypass xác thực và giới hạn kích thước	So trên routed path.
Token yếu, không giới hạn thử sai	Đoán token	Token ≥ 32 ký tự; 10 lần sai/60 s $\rightarrow$ 429 + Retry-After.
Route verify native trả 501	Không có đường xác minh đầu-cuối qua dịch vụ	Job verify với proof camera bắt buộc và root sổ đăng ký; trạng thái rejected riêng.
Job và upload không có hạn	Tích tụ dữ liệu, upload treo	Retention 24 h, artifact 10 phút, hạn upload qua biến môi trường.
Bộ đọc stdin bậc hai	Live embed chậm theo kích thước NAL	Compaction mỗi NAL, quét tiếp từ scan_; 1080p live embed 30,1 s $\rightarrow$ 2,5 s (đo 02/10, giao thức v2).
Kích thước SPS và file không giới hạn	SPS độc hại gây cấp phát lớn	36 864 MB, 543 MB/chiều; file 1 GiB; inspect 64 MiB.
Đường dẫn Unicode/dài trên Windows	CLI lỗi trong thư mục có tên tiếng Việt	wmain + UTF-8 + tiền tố đường dẫn dài.
Demo tách thành nhiều script	Khó duy trì, lặp mã	Một terminal_demo.py với deep trace đối chiếu FFmpeg.

13 Contract giữa các bước: input, output và lỗi

Ranh giới	Input	Output	Lỗi riêng
Digest video	Annex-B + K + frame_bits + max_bits_per_IDR	Digest 32 byte (cover = stego)	Ngoài profile, lịch thiếu dung lượng → video_digest_unavailable.
ZK bridge	Secret camera s + số đăng ký (đường Merkle) + binding	Proof + 3 public signal	Thiếu Node/circuit khác proof sai.
Nén proof	Proof dictionary / 129 byte	Bytes / proof đã kiểm điểm	ValueError khi điểm hoặc cò sai.
pack/unpack	Mode + message + proof	[0x01][mode][len 2B][message][proof]	Format, mode, độ dài sai → malformed_proof_payload.
Native encoder	Annex-B + K + payload + max_bits_per_IDR	Stego Annex-B (khung v3)	Cú pháp ngoài profile, thiếu dung lượng (mã 2).
Native decoder	Stego + cùng K + giới hạn payload + cùng max_bits_per_IDR	Payload ứng viên ("verified": false)	Version/độ dài sai, video kết thúc sớm (mã 2) → payload_not_found.
Groth16 verify	Proof + root + binding	True chỉ khi exit 0 và OK!	proof_invalid khác lỗi hạ tầng.
Job API	Upload + Bearer + trường form	Job id, trạng thái, artifact	401, 413, 429, 503; rejected khác failed.

14 Giới hạn còn lại

- **Luồng live chỉ ràng buộc message.** Ở mode = 1 proof không gắn với video vì phần còn lại của video chưa tồn tại khi proof được nhúng; job verify từ chối chế độ này theo mặc định. Hướng xử lý (commitment theo đoạn, chuỗi phiên, final seal) nằm trong future_plan.md.
- **Trusted setup là nghi thức demo.** Phase 1 công khai, nhưng hai lượt đóng góp Phase 2 và beacon chạy trên một máy; triển khai cần nhiều bên độc lập.
- **Proof không chứng minh cảnh thật.** Camera trong sổ đăng ký có thể quay lại một màn hình; proof chỉ nói camera đó xác nhận video và message.
- **Chỉ người có khóa giấu tin kiểm chứng được.** Digest phụ thuộc vị trí carrier, tức phụ thuộc K ; bên kiểm chứng vốn cần K để trích, nhưng hệ thống chưa có kiểm chứng cho bên không giữ khóa.
- **Keystream không có nonce.** Whitening chỉ phụ thuộc K và chỉ số bit, nên dùng lại cùng K cho nhiều video lặp lại dòng khóa.
- **Chưa có chứng minh không phát hiện được.** Chỉ đo thống kê tỉ lệ dấu ($|z| \leq 0,22$); steganalysis bằng học máy chưa được đo.
- **Realtime chưa được nghiệm thu.** Lần đo camera thật đã sửa cách đếm frame nguồn (run 24, ngày 24/09/2026) đạt 7,608 FPS so với công 30 FPS. Đây là số liệu lịch sử: đo với giao thức v1, trước đợt tăng tốc native ngày 02/10/2026, và chưa được đo lại; chưa có kết quả trên thiết bị edge.
- **Profile H.264 bị giới hạn.** Chỉ Baseline, CAVLC, progressive, 4:2:0, một slice mỗi frame bắt đầu ở MB 0, không I_PCM, một SPS/PPS id; chỉ IDR mang dữ liệu. Đây không phải hỗ trợ H.264 tổng quát.
- **Mong manh theo thiết kế.** Mọi lần mã hóa lại đều xóa payload (ca transcode → payload_not_found); hệ thống không phải watermark bền vững.
- **Không tương thích ngược.** Giao thức v3 không đọc được stego v2/v1; proof tạo bằng khóa mạch cũ không verify với khóa hiện tại.
- **Trust và chính sách khóa chưa được cứng chế ở đường native.** src/key_policy.py, src/manifest.py và src/trust/ (gồm MockTEESigner dùng HMAC) là giao diện thử nghiệm; CLI native và dịch vụ nhận khóa giấu tin thô, không tự kiểm chứng chỉ hay hạn khóa. Bearer token của API, khóa giấu tin và secret camera là ba credential khác nhau.
- **Triển khai một tiến trình.** Hàng đợi job nằm trong tiến trình và có lease thư mục làm việc, nên dịch vụ phải chạy với một worker Uvicorn.

15 Kết luận

Tại ngày 05/10/2026, VideoLevel là một prototype nghiên cứu có một lõi media native C++, một giao thức kênh v3 (HKDF, khung 0x03 không MAC, whitening) dùng chung cho file, HTTP và live, và một proof camera Groth16 (8 737 constraint, 3 public input) chứng minh một camera trong sổ đăng ký Merkle Poseidon xác nhận đúng video và message mà bên kiểm

chúng không cần secret camera. Xác minh fail-closed và là bước duy nhất xác thực payload; dịch vụ HTTP/WebSocket đã được làm cứng ở ranh giới xác thực và tài nguyên. Bộ kiểm thử 12 phase qua 142/142 (CTest 1/1), benchmark 04/10/2026 cho media 27/27, E2E 5/5 và 11/11 ca tấn công đúng kỳ vọng, cùng demo đối chiếu từng bước với FFmpeg, cung cấp bằng chứng chức năng trên host phát triển.

Không được suy ra rằng: proof chứng minh camera thật sự quay cảnh đó; luồng live được ràng buộc với video; payload trích ra là đáng tin trước khi Groth16 verify; hệ thống đạt realtime; giấu tin không phát hiện được; mọi profile H.264 được hỗ trợ; hoặc tham số Groth16 demo dùng được cho triển khai. Ưu tiên tiếp theo, theo `future_plan.md`: ràng buộc luồng live theo đoạn và phiên, nghi thức Phase 2 nhiều bên, đo lại realtime với giao thức v3 trên camera thật và thiết bị triển khai, và đánh giá steganalysis.